Métodos de Autenticación

- **Métodos de Autenticación**
  - **Básica**
  - **Digestión**
  - **API Key**
  - **Bearer Token**
  - **Json Web Token (JWT)**
  - **OAuth**
  - **Sesión**

b

Básica

Es el método mas sencillo de implementar y ampliamente soportado por servidores y clientes.
El usuario y password son transmitidos en los encabezados Http / Https codificados como Base64
El servidor decodifica y valida las credenciales para permitir el acceso.

- Cadena conteniendo credenciales : **`usuario:password`**
- Cadena convertida a Base64 : **`dXN1YXJpbzpwYXNzd29yZA==`**
- Encabezado en la petición Http : **`Authorization: Basic dXN1YXJpbzpwYXNzd29yZA==`**

Ventanas	Desventajas
Simplicidad	Usuario y password en riego
Fácil de implementar	No expira

Usos

- Entornos corporativos internos
- Desarrollos temporales y de prueba

Digestión

En este método de autenticación el cliente debe demostrar que conoce el password pero sin enviarlo. En lugar de enviarlo, envía un hash, aquí el servidor impone un desafío y el cliente debe demostrar que conoce el secreto sin revelarlo.

1. Cliente pide recurso protegido y servidor responde con encabezado:
   `WWW-Authenticate: Digest`
2. Cliente calcula hash y envía: **`Authorization: Digest`**
3. Servidor recalcula y compara hashes
   Si coinciden? **`201-OK`**
   No coinciden: **`401 Not Authorized`**

Cliente solicita recurso

```
GET /ruta-protegida HTTP/1.1
Host: ejemplo.com
```

Servidor responde con desafío, no da el recurso demuestra primero que conoces la clave
te envío un nonce y las reglas del desafío , Servidor establece las condiciones,

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Digest
    realm="MiAPI",            <--- El area protegida
    nonce="abc123",           <--- ID único generado por el servidor
    qop="auth",               <--- Nivel de protección auth/aut-int
    algorithm=MD5,            <--- Algoritmo
    opaque="...."
```

Cliente acepta reglas (`realm nonce qop algoritmo`) y calcula hash con esas reglas

Hash1 = AlgoritmoHash(username:realm:password)
Hash2 = AlgoritmoHash(método:url)
Hash3 = AlgoritmoHash(Hash1:nonce:nc:cnounce:qop:Hash2)ó

```
Authorization: Digest
    username="juan",
    realm="MiAPI",
    nonce="abc123",
    uri="/recurso",
    response="f0a12b9c...",
    qop=auth,
    nc=00000001,
    cnonce="xyz987"
```

API Key

Es una medida de seguridad muy básica para prevenir accesos no autorizados a la API.
Util cuando no se requiere un esquema de seguridad tan complejo y no se comparten datos sensibles.
Consiste en generar una llave única para cada cliente de la API. El ciente deberá incluir esta llave
en sus peticiones para efectos autenticación y autorización a la API.

Aquí no hay que hacer una petición a una ruta de login y enviar credenciales. La Api-key se obtiene mediante un proceso de registro, la envía previamente el proveedor o se obtiene en las opciones de configuración del servicio, una vez obtenida, podemos tener acceso:

Cliente envía petición con la API key:

```
GET /clientes HTTP/1.1
Host: api.ejemplo.com
x-api-key: abc123
```

Aunque no es recomendable, la API key puede enviarse en la URL como Query String:

```
GET /clientes?x-api-key=abc123
```

No hay un formato estándar para generación de Api-Leys, lo recomendable es generar
al menos 32 bytes aleatorios y convertirlos a Base64.

Bearer Token

El cliente obtiene un token del servidor para después incluirlo en el header de autorización usando el esquema Bearer.
Este token le concederá permiso para acceder a un recurso protegido.
El token consiste en una cadena de caracteres aleatoria, ejemplo:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ikpva
```

El Bearer token lo podemos obtener de dos maneras

- Enviando credenciales a una ruta de login que validará y regresará el token
- Obtenerlo previamente con un formato de registro o en las opciones de configuración del servicio.

El Bearer token puede ser de dos tipos:

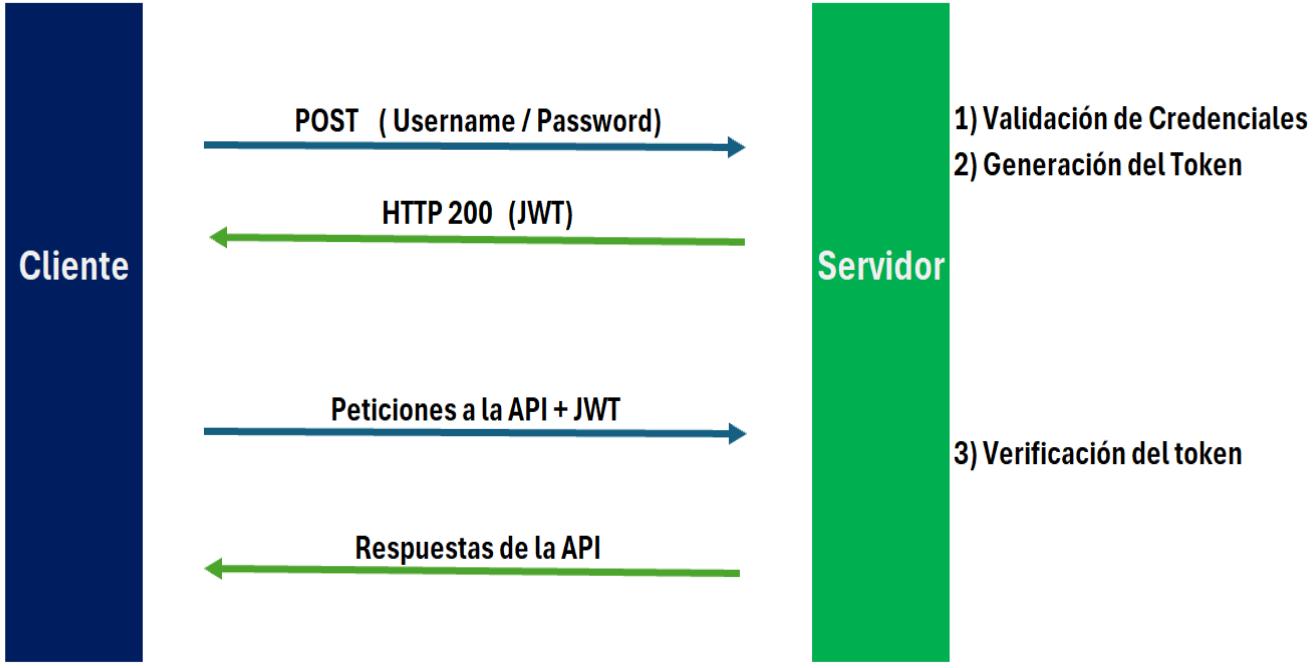
- **Ilegible**: No contiene información. También denominado opaco, o dicjo de otra manera es una cadena aleatoria y sin sentido.
- **Legible**: Contiene información, el estándar para estos es JWT pero existen otros menos usados como SAML y PASETO.

```
GET /clientes HTTP/1.1
Host: api.ejemplo.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ikpva
```

Json Web Token (JWT)

Es un método para transmitir información de manera segura entre dos partes.
Consiste en crear tokens de acceso (JWT) que serán usados para autenticar las solicitudes a la API.
Estos tokens de acceso contienen información del cliente, su expiración y firma criptográfica.

1. Cliente envía datos de autenticación a un servidor
2. Servidor valida los datos de autenticación y genera un `JWT`
3. El `JWT` se codifica en base64 y se envía al cliente
4. Peticiones subsecuentes del cliente al servidor, deberán contener el `JWT`.



OAuth

1. Usuario desea una nueva App y al accesar
   En lugar de crear una nueva cuenta da click en "Login with Google"

2. La aplicación re dirige al usuario hacia Google que a su vez
   le preguntará si quiere darle acceso a la App a su cuenta/perfil

3. Usuario da autorización y Google genera un código de autorización
   que le enviará a la app,

4. App intercambiará ese código de autorización por un token de acceso (JWT)
   con Google. Este token le proporciona a la App acceso limitado y seguro al perfil.

5. La App cpn el token de acceso, puede acceder a recursos del usuario
   generalmente solo verificar la identidad del usuario
   No tendrá acceso a las credenciales del usuario (login y password).

Sesión

Este es el método tradicional en cualquier aplicación web estándar. El cliente envía credenciales de acceso
(usuario y password) en una petición POST. El servidor valida las credenciales y si son correctas crea un ID de sesión.
Este ID de sesión se guarda en memoria, en una base de datos o en disco, con información de estado.
Servidor enviará ese ID al cliente para que lo incluya en peticiones subsecuentes

Cliente envía credenciales para iniciar sesión:
Las credenciales deben viajar en el body y siempre usando https.

```
POST /login
{
  "username": "juan",
  "password": "123456"
}
```

Servidor crea ID de sesión, y almacena la información de estado que necesite.
Responde con el encabezado **`Set Cookie`** con el id de sesión:
Información de estado almacenada en servidor:

```
session_id = "abc123"
session_data = {
  user_id: 42,
  nombre: "Gerardo",
  ultima_actividad: 2026-10-02 07:49
}
```

Encabezado en respuesta http, conteniendo id de sesión:

```
Set-Cookie: session_id=abc123; HttpOnly; Secure
```

La cookie debe tener estos atributos para protegerla de ataques:

- **`Httponly`**: No permite acceder la cookie desde Javascript (protege de XSS)
- **`Secure`**: La cookie solo se envía por Https
- **`SameSite`**: Evita que otros la usen (protege de CSRF)

Peticiones subsecuentes del cliente deberán llevar la sesión en el encabezado **`Cookie`**

```
GET /ruta
Cookie: session_idó=abc123
```

Ventajas	Desventajas
Control total en servidor	Servidor debe mantener sesiones en cache o disco
Fácil de implementar	Viola principio Rest de Sin Estado
El id de sesión no significa nada por si mismo	